1)Create a git repository clone it for changes and publish the changes using git bash.

- Install Git and open Git Bash.
- Create a new repository on GitHub.
- Copy the repository URL.
- Open Git Bash and navigate to the desired folder.
- Clone the repository using:
    - git clone <repository-url>
- Navigate into the cloned repository:
    - cd <repository-name>
- Create or modify a file in the repository.
- Check the changes:
    - git status
- Stage the changes:
    - git add .
- Commit the changes:
    - git commit -m "Updated files"
- Push the changes to GitHub:
    - git push origin main
- Open the GitHub repository and verify that the changes are published.

2) FIREBASE

STARTING

```
import { initializeApp } from "https://www.gstatic.com/firebasejs/13.0.0/firebase-app.js";

import { getDatabase, ref, set } from "https://www.gstatic.com/firebasejs/13.0.0/firebase-database.js";
```

ENDING

```
const db = getDatabase(app);

window.addStudent = function() {
  let roll = document.getElementById("roll").value.trim();
  let name = document.getElementById("name").value.trim();

  if (roll === "" || name === "") {
    alert("Enter roll number and student name");
    return;
  }

  set(ref(db, "students/" + roll), {
    rollNumber: roll,
```

```
      name: name
    })
    .then(() => {
      alert("Student Added Successfully");
    })
    .catch(error => {
      alert("Error: " + error.message);
    });
};
```

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Management System</title>
</head>
<body>
  <h2>Student Management System</h2>

  <input id="roll" placeholder="Roll Number">
  <input id="name" placeholder="Student Name">

  <button onclick="addStudent()">Add Student</button>

  <script type="module" src="p21.js"></script>
</body>
</html>
```

3) Explore Node.js Modules (os, http, etc.)

```
const os = require('os');
const fs = require('fs');
const http = require('http');

// OS Module
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("Hostname:", os.hostname());
console.log("OS Type:", os.type());
console.log("Release:", os.release());
console.log("Total Memory:", os.totalmem());
```

```
console.log("Free Memory:", os.freemem());
console.log("Home Directory:", os.homedir());
console.log("Temporary Directory:", os.tmpdir());
console.log("CPU Details:", os.cpus().length);

// FS Module
fs.writeFileSync('sample.txt', 'Hello Node.js!');
console.log("File created successfully.");

const data = fs.readFileSync('sample.txt', 'utf8');
console.log("File Content:", data);

fs.appendFileSync('sample.txt', '\nWelcome to File System Module!');
console.log("Data appended successfully.");

console.log("File Exists:", fs.existsSync('sample.txt'));

// HTTP Module
const server = http.createServer((req, res) => {
  res.write('Helloo');
  res.end();
});

server.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

EXECUTION :

- Run node app.js.
- Open http://localhost:3000 in your browser.

4) and 5) Create a REST API and Perform CRUD Operations

```
>npm init -y
>npm install express
>node app.js
```

```
const express = require('express');
const app = express();

app.use(express.json());

let students = [
```

```
  { id: 1, name: "Ravi" },
  { id: 2, name: "Priya" }
];

// Read students
app.get('/students', (req, res) => {
  res.json(students);
});

// Create student
app.post('/students', (req, res) => {
  students.push(req.body);
  res.send("Student added successfully");
});

// Update student
app.put('/students/:id', (req, res) => {
  const student = students.find(s => s.id == req.params.id);

  if (student) {
    student.name = req.body.name;
    res.send("Student updated successfully");
  } else {
    res.send("Student not found");
  }
});

// Delete student
app.delete('/students/:id', (req, res) => {
  students = students.filter(s => s.id != req.params.id);
  res.send("Student deleted successfully");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

- Open Postman and use http://localhost:3000/students.
- Select GET to view students.
- Select POST to add a student with JSON body, for example:

```
{
"id": 3,
```

```
"name": "Anu"
}
```

- Select PUT with http://localhost:3000/students/3 to update a student.
- Select DELETE with http://localhost:3000/students/3 to delete a student.

6) Create Authorized Endpoints Using JWT

```
const express = require('express');
const jwt = require('jsonwebtoken');

const app = express();
app.use(express.json());

const secret = 'mysecret';

// Login and generate token
app.post('/login', (req, res) => {
  const token = jwt.sign(
    { username: req.body.username },
    secret
  );
  res.json({ token });
});

// Verify token
function verifyToken(req, res, next) {
  const token = req.headers['authorization'];

  if (!token)
    return res.send("Access denied");

  try {
    req.user = jwt.verify(token, secret);
    next();
  } catch (err) {
    res.send("Invalid token");
  }
}

// Protected endpoint
app.get('/students', verifyToken, (req, res) => {
  res.json([
```

```
    { id: 1, name: "Ravi" },
    { id: 2, name: "Priya" }
  ]);
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

- Install packages - npm install express jsonwebtoken
- Run node app.js.
- Open Postman and send a POST request to http://localhost:3000/login.
- Select Body → raw → JSON and enter:

```
{
  "username": "admin"
}
```

- Copy the token from the response.
- Send a GET request to http://localhost:3000/students.
- Under Headers, add Authorization as the key and paste the token as its value.
- Click Send to view the student details. Without a valid token, access is denied.

7)react application using create-react-app and display "Hello-world"

```
function App() {
 return (
   <header>
    <h1>Hello World</h1>
   </header>
 );
}
export default App;
```

7)react application using create-react-app and display current date and time

```
function App() {
 const x=new Date();
 return (
   <header>
    <h1>Current Date and Time</h1>
    <h1>({x.toString()})</h1>
   </header>
 );
```

```
}
export default App;
```

9) Functional and class components

```
//functional – App.js
import React from "react";
function Student(props){
 return(
  <div>
   <h1> This is from Functional Component</h1>
   <p> Name: {props.name}</p>
   <p> Age:{props.age}</p>
  </div>
 )
}
function App() {
 return (
  <div>
   <Student name="Laasya" age="20"/>
  </div>
 );
}

export default App;

////class – CApp.js
import React,{Component} from "react";
class Student extends Component{
  render()
  {
    return(
      <div>
        <h1>This is from Class COmponent</h1>
        <p>Name:{this.props.name}</p>
        <p>Age:{this.props.age}</p>
      </div>
    )
  }
}
function CApp()
{
  return (
```

```
      <div>
        <Student name="Laasya" age="20"/>
      </div>
    )
}
export default CApp;
```

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import CApp from './CApp';
import reportWebVitals from './reportWebVitals';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
    <CApp/>
  </React.StrictMode>
);
reportWebVitals();
```

10) Create a class component that uses “state” to display Current date

App.js

```
import React,{Component} from "react";
class DateDisplay extends Component{
   constructor(props)
   {
      super(props);
      this.state={
         date:new Date()
      };
   }
   render()
   {
      return(
         <div>
            <h1>Current Date</h1>
            <h2>{this.state.date.toString()}</h2>
         </div>
```

```
      );
   }
}
function CApp()
{
   return(
      <DateDisplay/>
   );
}
export default CApp;
```

11) React application to demonstate the component lifecycle

```
import React, { Component } from "react";

class App extends Component {
 constructor() {
  super();
  this.state = { count: 0 };
  console.log("Constructor called");
 }

 componentDidMount() {
  console.log("Component Mounted");
 }

 componentDidUpdate() {
  console.log("Component Updated");
 }

 componentWillUnmount() {
     console.log("Component Unmounted");
  }

 render() {
  return (
   <div>
     <h1>React Lifecycle</h1>
     <h2>Count: {this.state.count}</h2>

     <button onClick={() => this.setState({ count: this.state.count + 1 })}>
      Increase
     </button>
```

```
    </div>
   );
  }
}

export default App;
```

12)React application to create 2 buttons and handle the events

```
import React, { useState } from "react";

function App() {
 const [message, setMessage] = useState("");

 function showDate() {
  setMessage(new Date().toLocaleString());
 }

 function showBye(){
  setMessage("Good Bye");
 };

 return (
  <div>
   <h1>Event Handling</h1>

   <button onClick={showDate}>Show Date and Time</button>

   <button onClick={showBye}>Good Bye</button>

   <h2>{message}</h2>
  </div>
 );
}

export default App;
```

13) CALCULATOR APPLICATION

```
import React, { useState } from 'react';
function App() {
 const [v, setV] = useState('');
```

```
  return (<>
     <style>{`
      .calc { width: 250px;
       margin: 50px auto;
      text-align: center; }
      button { padding: 10px;
      margin: 3px; }
     `}</style>

     <div className="calc">
      <h2>Calculator</h2>
      <input value={v} readOnly />
      <br />
      <button onClick={() => setV(v + '7')}>7</button>
      <button onClick={() => setV(v + '8')}>8</button>
      <button onClick={() => setV(v + '9')}>9</button>
      <button onClick={() => setV(v + '/')}>/</button>
      <br />

      <button onClick={() => setV(v + '4')}>4</button>
      <button onClick={() => setV(v + '5')}>5</button>
      <button onClick={() => setV(v + '6')}>6</button>
      <button onClick={() => setV(v + '*')}>*</button>
      <br />

      <button onClick={() => setV(v + '1')}>1</button>
      <button onClick={() => setV(v + '2')}>2</button>
      <button onClick={() => setV(v + '3')}>3</button>
      <button onClick={() => setV(v + '-')}>-</button>
      <br />
      <button onClick={() => setV(v + '0')}>0</button>
      <button onClick={() => setV(v + '+')}>+</button>
      <button onClick={() => setV(String(eval(v)))}>=</button>
      <button onClick={() => setV('')}>C</button>
     </div>
   </>
  );
}

export default App;
```

14)React application to create a registration form

```
import React, { useState } from "react";

function App() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  function register(e) {
    e.preventDefault();
    alert("Registration Successful!");
  }

  return (
    <div>
      <h1>Student Registration</h1>

      <form onSubmit={register}>
        <input
          type="text"
          placeholder="Enter Name"
          value={name}
          onChange={e => setName(e.target.value)}
          required
        />

        <br /><br />

        <input
          type="email"
          placeholder="Enter Email"
          value={email}
          onChange={e => setEmail(e.target.value)}
          required
        />

        <br /><br />

        <input
          type="password"
          placeholder="Enter Password"
          value={password}
```

```
        onChange={e => setPassword(e.target.value)}
        required
        minLength={6}
      />

      <br /><br />

      <button type="submit">Register</button>
    </form>
  </div>
 );
}

export default App;
```